

Plataforma local de agentes y automatizaciones

Arquitectura, seguridad y hoja de ruta para construir un asistente personal con voz, correo, desarrollo de software e Infrastructure as Code.

Hardware inicial

**Intel Core i7-8700K | 32 GB RAM |
NVIDIA GTX 1060 6 GB**

Principio rector

El LLM propone. Las herramientas verifican. La persona autoriza.

Resumen ejecutivo

El equipo actual es suficiente para ejecutar una primera plataforma local útil. Puede sostener n8n, PostgreSQL, un modelo de lenguaje cuantizado, reconocimiento y síntesis de voz, integraciones de correo y ejecutores aislados para tareas de código.

La restricción principal es la VRAM: conviene mantener un único modelo activo, procesar las tareas pesadas de forma secuencial y reservar las decisiones sensibles para validadores deterministas y aprobación humana.

Conclusión: el proyecto es viable ahora. La mejora futura de hardware aumentará velocidad y contexto, pero no debería cambiar la arquitectura de seguridad ni los contratos entre componentes.

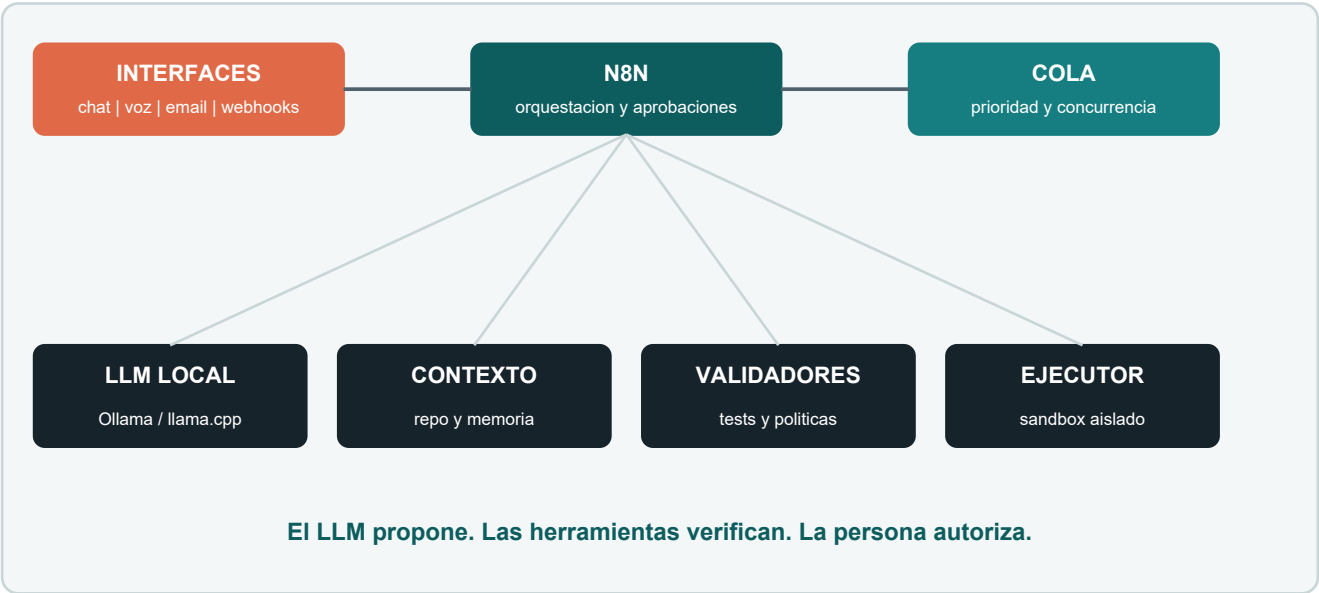
Capacidad esperable

Área	Viabilidad actual	Observación
n8n y automatizaciones	Alta	Carga moderada; ejecutar permanentemente.
Correo y resúmenes	Alta	Usar OAuth y acceso read-only al comenzar.
Chat textual	Alta	Un modelo 7B cuantizado ofrece una experiencia útil.
Voz por turnos	Media-alta	Push-to-talk; habrá pausas perceptibles.
Código y Terraform	Media-alta	Trabajar por archivos y con validadores reales.
Conversación full-duplex	Baja-media	Streaming e interrupciones requieren más ingeniería.
Varios modelos simultáneos	Baja	Compartir un modelo entre roles y usar una cola.

Modelo operativo recomendado

- Qwen2.5-Coder 7B Q4 como modelo principal para conversación técnica, código y Terraform.
- Contexto operativo moderado, con recuperación selectiva de archivos en lugar de cargar repositorios completos.
- STT y TTS ejecutados como servicios separados para poder reemplazarlos sin modificar los workflows.
- Una sola inferencia pesada a la vez, administrada por prioridad.

Arquitectura de referencia



Responsabilidades

Capa	Responsabilidad
Interfaces	Chat, voz, email, Telegram o webhooks. No contienen lógica privilegiada.
n8n	Orquesta workflows, horarios, estados, aprobaciones y notificaciones.
LLM local	Interpreta solicitudes, propone planes, redacta y revisa. No es una frontera de seguridad.
Contexto	Selecciona AGENTS.md, documentación, archivos y memoria necesarios para cada tarea.
Validadores	Aplican schemas, tests, linters, políticas, análisis de planes y escaneo de secretos.
Ejecutor	Modifica repositorios y ejecuta herramientas dentro de un entorno aislado.

Distribución sobre el equipo actual

- GTX 1060 6 GB
- LLM 7B cuantizado, un trabajo a la vez
 - STT acelerado solo cuando la cola lo permita
- i7-8700K + 32 GB RAM
- n8n, PostgreSQL, TTS y automatizaciones
 - indexación, Terraform, tests y scanners
 - offload parcial de modelos que excedan la VRAM

Prioridad sugerida: voz > interacción manual > alertas > correo > código e IaC programados.

Catálogo de agentes

Los agentes son roles, no necesariamente procesos ni modelos diferentes. En el hardware actual, el mismo modelo puede asumir cada rol con un prompt, herramientas y permisos específicos.

Agente	Responsabilidad	Permiso máximo
Router	Clasificar intención y seleccionar workflow.	Lectura; sin herramientas sensibles.
Conversacional	Chat general y respuesta por voz.	Lectura y memoria limitada.
Correo	Clasificar y resumir mensajes.	OAuth read-only.
Investigación	Buscar, comparar y citar fuentes.	Internet read-only y dominios permitidos.
Analista de repositorio	Inventariar código, dependencias y convenciones.	Repositorio read-only.
Arquitecto	Proponer diseño, costos y riesgos.	Sin escritura ni ejecución.
Implementador	Modificar código según un plan aprobado.	Rama o worktree aislado.
Especialista IaC	Crear y revisar Terraform.	Rama; nunca apply autónomo.
Revisor	Examinar diff, tests y regresiones.	Solo lectura.
Seguridad	Detectar secretos, IAM excesivo y superficies de ataque.	Solo lectura y scanners.
Costos	Contrastar cuotas, presupuesto y plan.	APIs cloud read-only.
Operaciones	Resumir alertas e incidentes.	Observabilidad read-only.
Despliegue	Invocar operaciones predefinidas.	Wrapper y allowlist estrictos.

Un prompt de sistema orienta el comportamiento, pero no impide técnicamente una acción. Los permisos efectivos deben imponerse mediante OAuth scopes, contenedores, wrappers, allowlists, políticas y credenciales separadas.

Workflows principales

1. Conversación

mensaje -> router -> contexto -> agente -> validación -> respuesta

Control principal: El router no recibe permisos para ejecutar acciones.

2. Voz

push-to-talk -> STT -> router -> agente -> TTS -> audio

Control principal: La voz tiene prioridad, pero comparte el mismo sistema de autorización que el chat.

3. Resumen de correo

scheduler -> OAuth -> sanitizar -> clasificar -> resumir -> digest

Control principal: El contenido del email es entrada no confiable y nunca puede redefinir instrucciones.

4. Desarrollo de software

analizar -> planificar -> crear rama -> editar -> probar -> revisar -> aprobar

Control principal: El ejecutor trabaja aislado y no accede al Docker socket ni al host completo.

5. Infrastructure as Code

diseño -> aprobación -> implementar -> validate -> plan JSON -> políticas -> revisión

Control principal: Bloqueo automático ante secretos, costos inciertos, delete, replace o IAM amplio.

6. Operaciones

alerta -> correlacionar -> diagnosticar -> proponer -> notificar -> aprobar

Control principal: Al principio solo diagnostica; reinicios y restauraciones requieren aprobación.

Seguridad y niveles de confianza

Nivel	Capacidad	Ejemplo	Aprobación
L0	Leer y responder	Consultar documentación	No
L1	Crear borradores	Preparar respuesta de correo	Antes de enviar
L2	Escribir aislado	Crear rama y modificar Terraform	Antes de merge
L3	Operación predefinida	Desplegar imagen autorizada	Según política
L4	Producción/destructivo	apply, borrar o restaurar	Siempre

Reglas transversales

- Separar credenciales por agente, aplicación y dominio de confianza.
- Guardar secretos en n8n o en un gestor de secretos, nunca en Git ni en prompts.
- Registrar actor, solicitud, herramientas invocadas, resultado y aprobación.
- Aplicar schemas a entradas y salidas; rechazar campos desconocidos en acciones sensibles.
- Usar allowlists para repositorios, comandos, imágenes, rutas y operaciones desplegables.
- No montar /var/run/docker.sock dentro de n8n ni dentro de un agente generalista.
- Tratar páginas web, emails, issues y documentos como contenido no confiable.
- Mantener terraform apply en un workflow separado y desactivado por defecto.

Contrato de un despliegue restringido

Permitido:

```
deploy-app <app-allowlisted> <commit-sha>
rollback-app <app-allowlisted>
deployment-status <app-allowlisted>
```

Prohibido:

```
comandos Docker arbitrarios, Compose aportado por el pipeline,
rutas libres, parámetros extra y credenciales persistentes.
```

Repositorio de la plataforma

Conviene mantener la plataforma de agentes en un repositorio separado de los proyectos que inspecciona. Así, la automatización puede evolucionar sin mezclar sus credenciales, runners o workflows con la IaC de OCI.

```
personal-agent-platform/  
  compose.yml  
  agents/      # roles, permisos y herramientas  
  prompts/     # instrucciones versionadas  
  schemas/     # contratos JSON de entrada/salida  
  workflows/   # voz, correo, desarrollo, IaC, operaciones  
  policies/    # Terraform, IAM, costos, despliegues  
  runners/     # ejecución aislada  
  integrations/ # correo, Git, OCI, notificaciones  
  observability/ # logs, métricas y auditoría  
  tests/  
  AGENTS.md  
  README.md
```

Validación determinista

Dominio	Validaciones
Respuestas LLM	JSON Schema, campos obligatorios, límites y enumeraciones.
Código	Tests, linters, build, análisis estático y revisión del diff.
Terraform	fmt, init -backend=false, validate, plan -json, políticas y diff check.
Seguridad	Secret scanning, dependencias, IAM, rutas, permisos y exposición de red.
Costos	Cuotas reales, fuentes oficiales, recursos existentes y presupuesto máximo.
Despliegues	Repositorio permitido, SHA inmutable, digest, health check y rollback.

No usar otro LLM como único validador. Un segundo agente ayuda a encontrar errores, pero las garantías deben provenir de herramientas y políticas reproducibles.

Hoja de ruta

Fase	Trabajo	Resultado
1. Base	n8n, PostgreSQL, Ollama, HTTPS/VPN, backups y monitoreo.	Plataforma estable y recuperable.
2. Chat	Interfaz textual, router simple y respuestas estructuradas.	Conversación local funcional.
3. Correo	OAuth read-only, digest y protección contra prompt injection.	Resumen diario confiable.
4. Voz	Push-to-talk, STT, TTS y prioridad de cola.	Asistente de voz por turnos.
5. Desarrollo	Worktrees, runner aislado, tests y revisión.	Cambios en ramas sin tocar producción.
6. IaC	Terraform, políticas, plan JSON, costos y bloqueos.	Propuestas verificables sin apply.
7. Operaciones	Alertas, logs, despliegues restringidos y auditoría.	Automatización operativa controlada.
8. Separación	ThinkCentre como control y PC GPU como worker.	Menor consumo y mejor disponibilidad.

Orden recomendado

Cada fase debe producir métricas, pruebas y un procedimiento de recuperación antes de habilitar la siguiente. La voz y los agentes de código se benefician de una base estable, pero no deberían retrasar el primer caso útil: el resumen de correo read-only.

Criterios para avanzar

- El servicio reinicia sin intervención y conserva datos.
- Existe backup probado de PostgreSQL y configuración.
- Las credenciales están cifradas y fuera de Git.
- Cada workflow tiene timeout, reintentos limitados y ruta de error.
- Las acciones sensibles generan una solicitud explícita de aprobación.
- El sistema registra suficiente información para explicar qué ocurrió.

Primer objetivo práctico

Completar un flujo de extremo a extremo: recibir una pregunta escrita, enrutarla al modelo local, devolver una respuesta estructurada y registrar la ejecución. Luego reutilizar esa base para correo y voz.

Evolución del hardware

Etapa actual

- PC actual
- n8n + PostgreSQL
 - Ollama / llama.cpp
 - STT + TTS
 - runners y validadores
 - un trabajo de inferencia pesado por vez

Etapa futura

- ThinkCentre siempre encendido
- n8n, PostgreSQL, integraciones, colas, VPN y monitoreo
- PC con GPU bajo demanda
- LLM, STT y agentes de código
 - trabajador registrado con permisos limitados

Una GPU de 12 GB permitirá modelos 14B cuantizados, más contexto y mejor concurrencia. El ThinkCentre aportará disponibilidad y menor consumo. Ninguna de esas mejoras elimina la necesidad de aislamiento y aprobación.

Decisiones para el MVP

Decisión	Recomendación inicial
Sistema anfitrión	Linux dedicado; en Windows, usar WSL2 y guardar datos dentro del filesystem Linux.
Base de datos	PostgreSQL desde el inicio para una instancia permanente.
Vector database	No al comienzo; seleccionar archivos con rg y resúmenes jerárquicos.
Interfaz de voz	Push-to-talk antes de wake word o conversación full-duplex.
Exposición remota	VPN privada preferentemente; HTTPS y autenticación si se publica.
Concurrencia	Una cola y un único LLM pesado activo.

Checklist de inicio

Infraestructura

- Docker Compose con versiones fijadas y volúmenes persistentes.
- n8n, PostgreSQL y Ollama en redes internas separadas.
- Backups automáticos con prueba periódica de restauración.
- Monitoreo de disco, memoria, temperatura, disponibilidad y fallos de workflows.
- Acceso remoto mediante VPN o proxy HTTPS endurecido.

Seguridad

- Clave de cifrado de n8n respaldada fuera del repositorio.
- OAuth de correo con el alcance mínimo posible.
- Runners sin acceso al host, al socket Docker ni a credenciales innecesarias.
- Separación entre lectura, escritura, despliegue y acciones destructivas.
- Aprobación humana para envíos, merges, despliegues y cambios cloud.

Calidad

- Prompts y schemas versionados.
- Casos de prueba con entradas válidas, ambiguas y maliciosas.
- Evaluación periódica de respuestas y falsos positivos.
- Registro de modelo, prompt, herramientas y duración por ejecución.
- Fallback claro cuando el modelo no puede verificar una afirmación.

Fuentes técnicas consultadas

n8n - instalación con Docker Compose:
<https://docs.n8n.io/deploy/host-n8n/install-options/install-using-docker-compose.md>

n8n - integración Ollama Chat Model:
<https://docs.n8n.io/integrations/builtin/cluster-nodes/sub-nodes/n8n-nodes-langchain.lmchatollama/>

Ollama - Qwen2.5-Coder y tamaños de modelos:
<https://ollama.com/library/qwen2.5-coder/tags>

whisper.cpp - reconocimiento de voz y soporte CUDA:
<https://github.com/ggml-org/whisper.cpp>

NVIDIA - especificaciones GTX 1060:
<https://www.nvidia.com/en-us/geforce/news/nvidia-geforce-gtx-1060/>